

Name: Eman Fatima
Reg No: BSSE-23S-0025
Program: BSSE
Date: August 31st, 2026

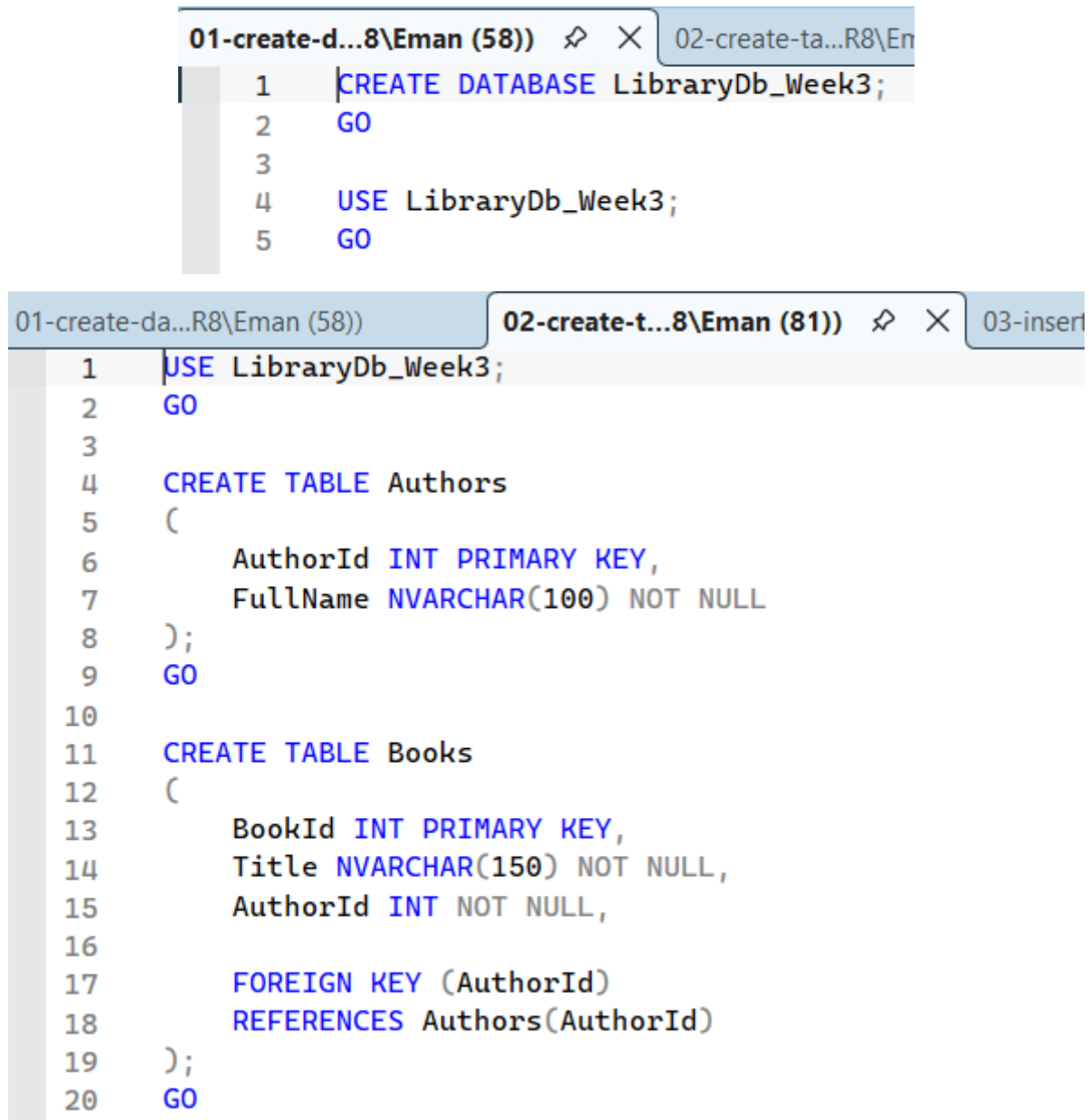
WEEK NO. 3-

PART A: RELATIONAL DATABASE DESIGN

Exercises:-

1. Create Authors and Books Tables

Code:



```
01-create-d...8\Eman (58)) 02-create-ta...R8\En
1 CREATE DATABASE LibraryDb_Week3;
2 GO
3
4 USE LibraryDb_Week3;
5 GO

01-create-da...R8\Eman (58)) 02-create-t...8\Eman (81)) 03-insert
1 USE LibraryDb_Week3;
2 GO
3
4 CREATE TABLE Authors
5 (
6     AuthorId INT PRIMARY KEY,
7     FullName NVARCHAR(100) NOT NULL
8 );
9 GO
10
11 CREATE TABLE Books
12 (
13     BookId INT PRIMARY KEY,
14     Title NVARCHAR(150) NOT NULL,
15     AuthorId INT NOT NULL,
16
17     FOREIGN KEY (AuthorId)
18     REFERENCES Authors(AuthorId)
19 );
20 GO
```

Output:

SQLQuery9....\Eman (53))* 01-create-da...R

```

1  USE LibraryDb_Week3;
2  GO
3
4
5  SELECT * FROM Authors
6  SELECT * FROM Books

```

100 % No issues found

Results Messages

	AuthorId	FullName
1	1	Robert C. Martin
2	2	Andrew Hunt
3	3	Tom Taulli

	BookId	Title	AuthorId
1	1	Clean Code	1
2	2	Clean Architecture	1
3	3	The Pragmatic Programmer	2
4	4	Programming Ruby	2
5	5	Artificial Intelligence Basics	3
6	6	AI for Business	3

2. Insert 3 Authors and 6 Books

Code:

01-create-da...R8\Eman (58) 02-create-ta...R8\Eman (81) 03-insert-d...8\Eman (82)) ✕

```

1  USE LibraryDb_Week3;
2  GO
3
4  INSERT INTO Authors (AuthorId, FullName)
5  VALUES
6  (1, 'Robert C. Martin'),
7  (2, 'Andrew Hunt'),
8  (3, 'Tom Taulli');
9  GO
10
11 INSERT INTO Books (BookId, Title, AuthorId)
12 VALUES
13 (1, 'Clean Code', 1),
14 (2, 'Clean Architecture', 1),
15 (3, 'The Pragmatic Programmer', 2),
16 (4, 'Programming Ruby', 2),
17 (5, 'Artificial Intelligence Basics', 3),
18 (6, 'AI for Business', 3);
19 GO
20

```

Output:

SQLQuery9....\Eman (53))* 01-create-da...R

```
1 USE LibraryDb_Week3;
2 GO
3
4
5 SELECT * FROM Authors
6 SELECT * FROM Books
```

100 % No issues found

Results Messages

	AuthorId	FullName
1	1	Robert C. Martin
2	2	Andrew Hunt
3	3	Tom Taulli

	BookId	Title	AuthorId
1	1	Clean Code	1
2	2	Clean Architecture	1
3	3	The Pragmatic Programmer	2
4	4	Programming Ruby	2
5	5	Artificial Intelligence Basics	3
6	6	AI for Business	3

3. Join Books With Authors

Code:

01-create-da...R8\Eman (58)) 02-create-ta...R8\Eman (81)) 03-insert-dat...R8\Eman (82)) 04-join-que...8\Eman (83))

```
1 USE LibraryDb_Week3;
2 GO
3
4 SELECT
5     b.Title,
6     a.FullName AS Author
7 FROM Books b
8 JOIN Authors a
9     ON b.AuthorId = a.AuthorId
10 ORDER BY a.FullName;
11 GO
```

Output:

91 % 6 0

Results Messages

	Title	Author
1	The Pragmatic Programmer	Andrew Hunt
2	Programming Ruby	Andrew Hunt
3	Clean Code	Robert C. Martin
4	Clean Architecture	Robert C. Martin
5	Artificial Intelligence Basics	Tom Taulli
6	AI for Business	Tom Taulli

4. Create Categories and BookCategories and Insert Many-to-Many Data

Code:

```
01-create-da...R8\Eman (58)) 02-create-t...8\Eman (81)) ✨ ×
22 CREATE TABLE Categories
23 (
24     CategoryId INT PRIMARY KEY,
25     CategoryName NVARCHAR(100) NOT NULL
26 );
27 GO
28
29 CREATE TABLE BookCategories
30 (
31     BookId INT NOT NULL,
32     CategoryId INT NOT NULL,
33
34     PRIMARY KEY (BookId, CategoryId),
35
36     FOREIGN KEY (BookId)
37     REFERENCES Books(BookId),
38
39     FOREIGN KEY (CategoryId)
40     REFERENCES Categories(CategoryId)
41 );
42 GO
```

```
01-create-da...R8\Eman (58)) 02-create-ta...R8\Eman (81)) 03-insert-d...8\Eman (82)) ✨ ×
20
21 INSERT INTO Categories (CategoryId, CategoryName)
22 VALUES
23 (1, 'Programming'),
24 (2, 'Software Engineering'),
25 (3, 'Artificial Intelligence'),
26 (4, 'Business');
27 GO
28
29 INSERT INTO BookCategories (BookId, CategoryId)
30 VALUES
31 (1, 1),
32 (1, 2),
33 (2, 1),
34 (2, 2),
35 (3, 1),
36 (3, 2),
37 (4, 1),
38 (5, 3),
39 (6, 3),
40 (6, 4);
41 GO
```

Output:

```
1  USE LibraryDb_Week3;
2  GO
3
4
5  SELECT * FROM Categories
6  SELECT * FROM BookCategories
```

91 % 2 0

Results Messages

	CategoryId	CategoryName
1	1	Programming
2	2	Software Engineering
3	3	Artificial Intelligence
4	4	Business

	BookId	CategoryId
1	1	1
2	1	2
3	2	1
4	2	2
5	3	1
6	3	2
7	4	1
8	5	3
9	6	3
10	6	4

5. Get Categories for One Book

Code:

01-create-da...R8\Eman (58))	02-create-ta...R8\Eman (81))	03-insert-dat...R8\Eman (82))	04-join-que...8\Eman (83))
<pre>13 SELECT 14 b.Title, 15 c.CategoryName 16 FROM Books b 17 JOIN BookCategories bc 18 ON b.BookId = bc.BookId 19 JOIN Categories c 20 ON bc.CategoryId = c.CategoryId 21 WHERE b.BookId = 1; 22 GO</pre>			

Output:

	Title	CategoryName
1	Clean Code	Programming
2	Clean Code	Software Engineering

6. Foreign Key Delete Test

Code:

```
05-delete-t...8\Eman (84))  
1  USE LibraryDb_Week3;  
2  GO  
3  
4  DELETE FROM Authors  
5  WHERE AuthorId = 1;  
6  GO
```

Output:

83 % No issues found

Messages

Msg 547, Level 16, State 0, Line 4
The DELETE statement conflicted with the REFERENCE constraint "FK_Books__AuthorId__4CA06962". The conflict occurred in database "LibraryDb_Week3", table "dbo.Books", column 'AuthorId'.
The statement has been terminated.

Completion time: 2026-09-08T21:49:20.8974887+05:00

Explanation:

The delete was rejected because books still reference that author. The foreign key protects the relationship and prevents invalid data.

PART B: ENTITY FRAMEWORK CORE

Exercises:-

1. Create Standalone EF Core Project and Create and Apply Migration

Code:

```
Models > Book.cs > {} EFCorePractice.Models > Book  
1  namespace EFCorePractice.Models;  
2  
3  public class Book  
4  {  
5      public int BookId { get; set; }  
6  
7      public string Title { get; set; } = string.Empty;  
8  
9      public string Author { get; set; } = string.Empty;  
10 }
```

```

Data > C# LibraryDbContext.cs > ...
1  using EFCorePractice.Models;
2  using Microsoft.EntityFrameworkCore;
3
4  namespace EFCorePractice.Data;
5
6  public class LibraryDbContext : DbContext
7  {
8      public LibraryDbContext(
9          DbContextOptions<LibraryDbContext> options)
10         : base(options)
11     {
12     }
13
14     public DbSet<Book> Books => Set<Book>();
15 }

```

Output:

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the Object Explorer shows the database structure for 'localhost (SQL Server 17.0.1000.7 - DESKTOP-IQSS2R8\)'.

- Object Explorer:**
 - localhost (SQL Server 17.0.1000.7 - DESKTOP-IQSS2R8\)
 - Databases
 - System Databases
 - Database Snapshots
 - EFCorePracticeDb
 - Database Diagrams
 - Tables
 - System Tables
 - FileTables
 - External Tables
 - Graph Tables
 - dbo.__EFMigrationsHistory
 - dbo.Books
 - Columns
 - Keys
 - Constraints
 - Triggers
 - Indexes
 - Statistics
 - Dropped Ledger Tables

On the right, the SQL Query window shows the following query:

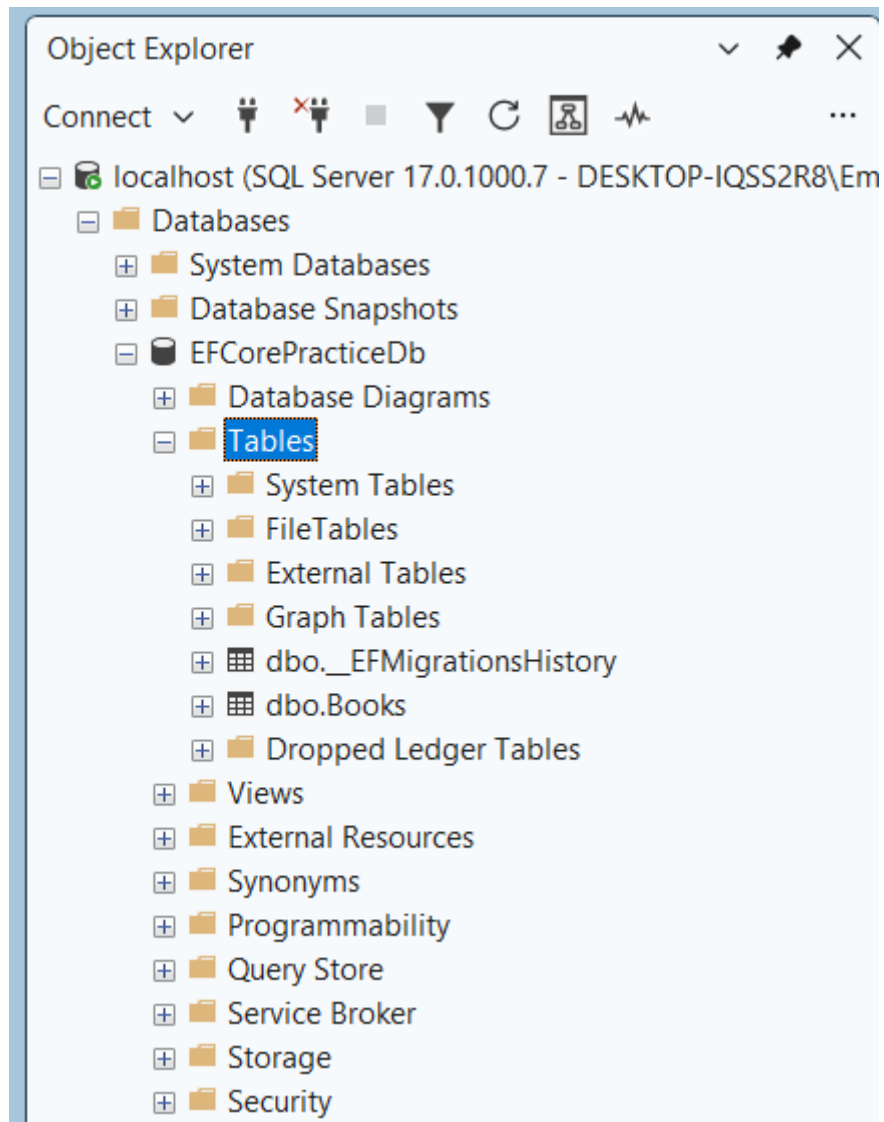
```

1  USE EFCorePracticeDb;
2  GO
3
4  SELECT * FROM Books;

```

The query results are displayed in a table with the following data:

	BookId	Title	Author
1	1	Clean Code - Updated	Robert C. Martin
2	2	The Pragmatic Programmer	Andrew Hunt



Commands:

- dotnet ef migrations add InitialCreate
- dotnet ef database update

2. Insert and Retrieve Records

Code:

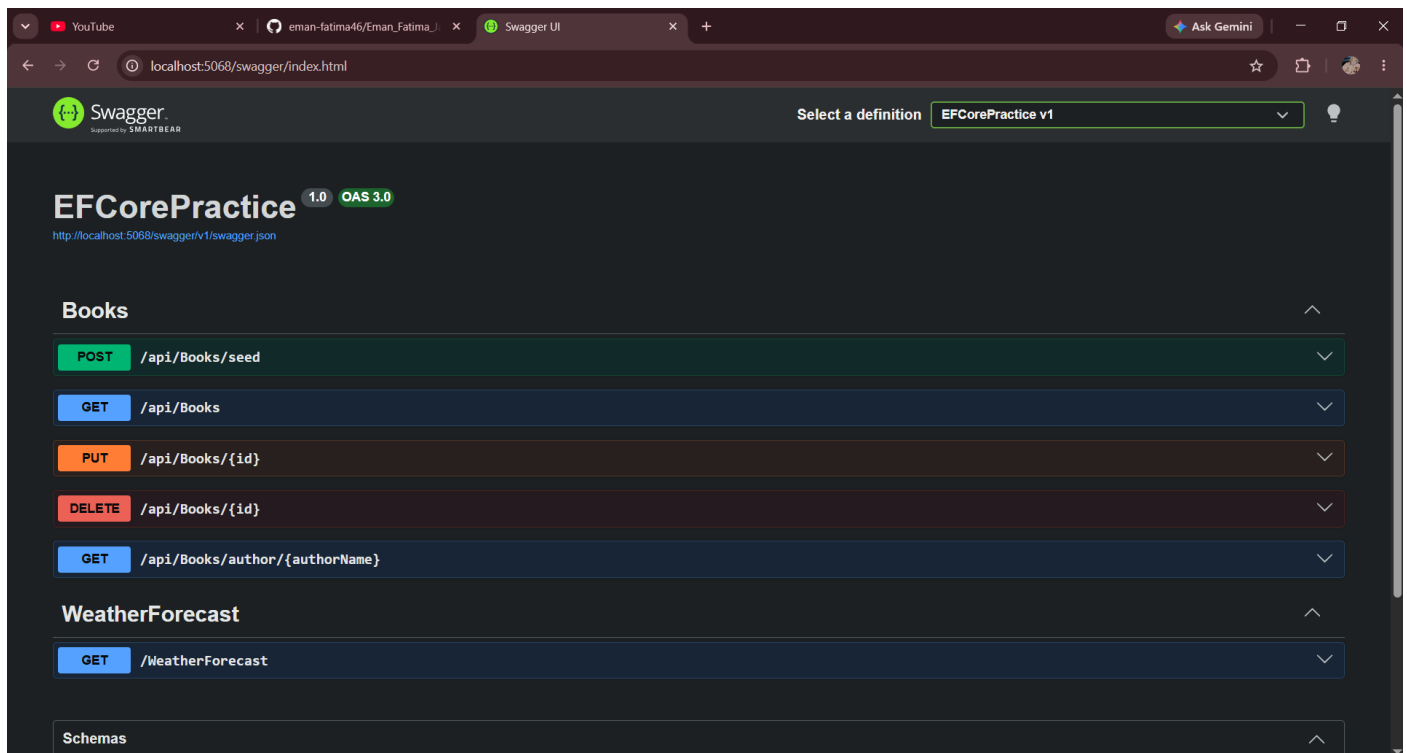
Controllers > C# BooksController.cs > {} EFCorePractice.Controllers > BooksController

```
6 namespace EFCorePractice.Controllers;
10 public class BooksController : ControllerBase
22     public async Task<IActionResult> AddSampleBooks()
23     {
24         if (await _context.Books.AnyAsync())
25         {
26             return BadRequest(
27                 "Books already exist."
28             );
29         }
30
31         _context.Books.Add(
32             new Book
33             {
34                 Title = "Clean Code",
35                 Author = "Robert C. Martin"
36             }
37         );
38
39         _context.Books.Add(
40             new Book
41             {
42                 Title = "The Pragmatic Programmer",
43                 Author = "Andrew Hunt"
44             }
45         );
46
47         _context.Books.Add(
48             new Book
49             {
50                 Title = "AI Basics",
51                 Author = "Tom Taulli"
52             }
53         );
54
55         await _context.SaveChangesAsync();
56
57         return Ok(
```

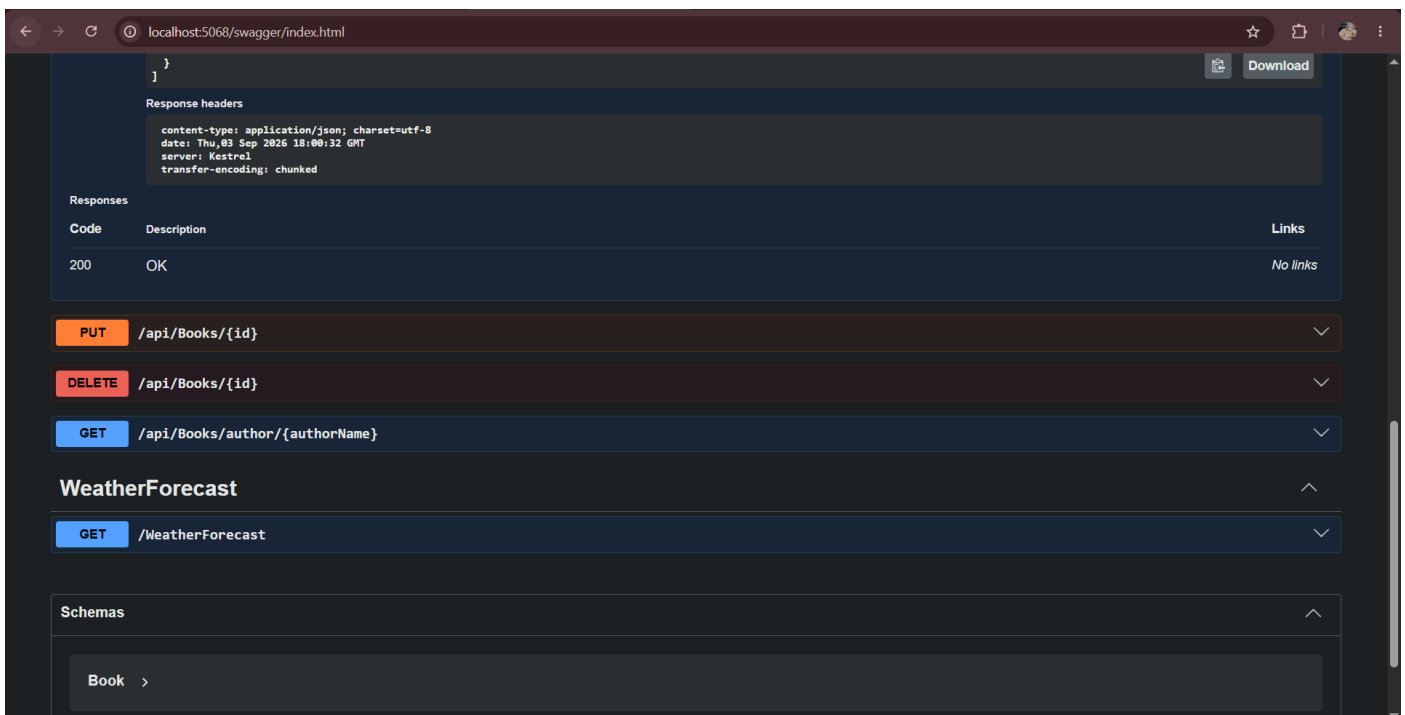
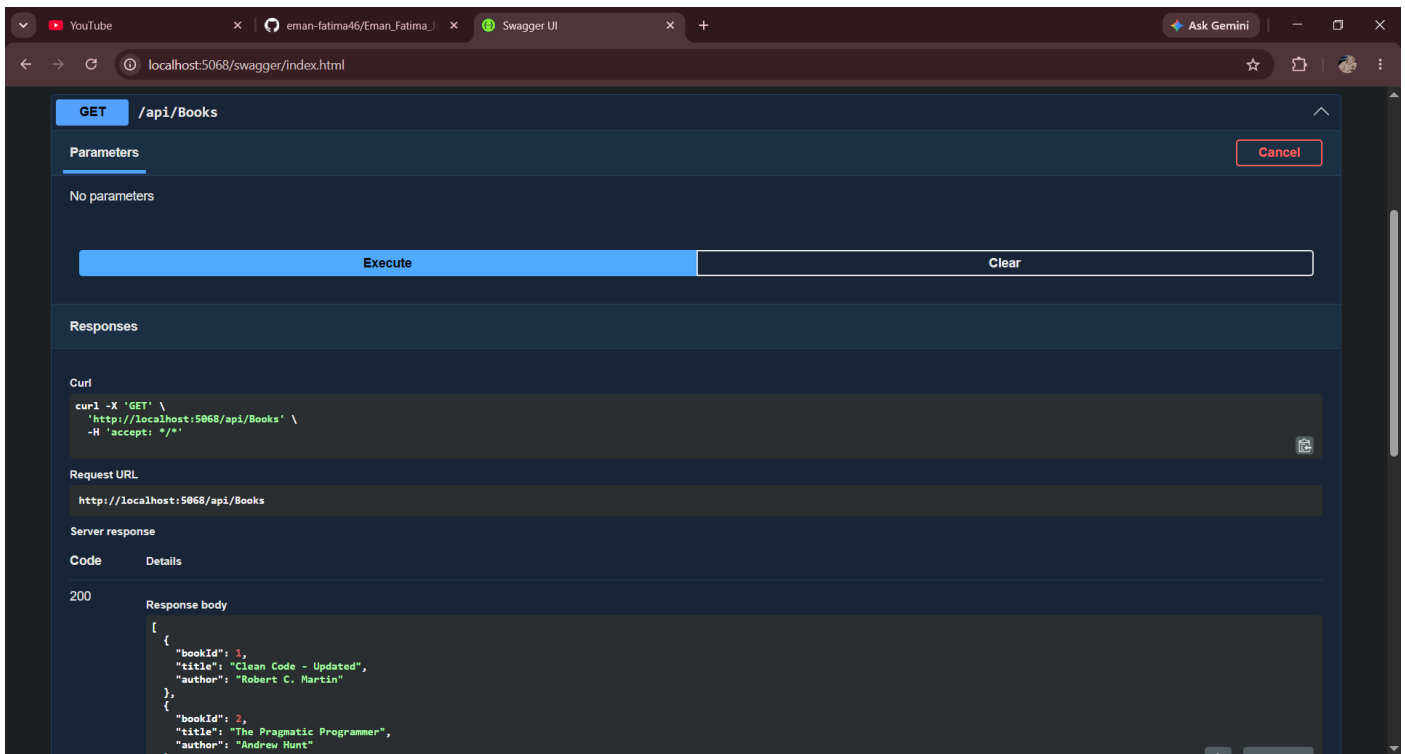
Controllers >  BooksController.cs > {} EFCorePractice.Controllers >  BooksController

```
6 namespace EFCorePractice.Controllers;
10 public class BooksController : ControllerBase
22     public async Task<IActionResult> AddSampleBooks()
58         "Sample books inserted."
59     );
60 }
61
62
63 [HttpGet]
0 references
64 public async Task<IActionResult> GetBooks()
65 {
66     List<Book> books =
67         await _context.Books.ToListAsync();
68
69     return Ok(books);
70 }
```

Output:



The screenshot displays the Swagger UI for an API named 'EFCorePractice' (version 1.0, OAS 3.0). The browser tabs show 'YouTube', 'eman-fatima46/Eman_Fatima_J', and 'Swagger UI'. The address bar indicates the URL 'localhost:5068/swagger/index.html'. The Swagger UI header includes the 'Swagger' logo and a 'Select a definition' dropdown set to 'EFCorePractice v1'. The API endpoints are organized into two main sections: 'Books' and 'WeatherForecast'. The 'Books' section lists five endpoints: POST /api/Books/seed, GET /api/Books, PUT /api/Books/{id}, DELETE /api/Books/{id}, and GET /api/Books/author/{authorName}. The 'WeatherForecast' section lists one endpoint: GET /WeatherForecast. A 'Schemas' section is also visible at the bottom.



3. Update and Delete a Record

Code:

Controllers > BooksController.cs > {} EFCorePractice.Controllers > BooksController

```
6 namespace EFCorePractice.Controllers;
10 public class BooksController : ControllerBase

73 [HttpPut("{id}")]
    0 references
74 public async Task<IActionResult> UpdateBook(
75     int id,
76     Book updatedBook)
77 {
78     Book? book =
79         await _context.Books.FindAsync(id);
80
81     if (book == null)
82     {
83         return NotFound();
84     }
85
86     book.Title = updatedBook.Title;
87     book.Author = updatedBook.Author;
88
89     await _context.SaveChangesAsync();
90
91     return Ok(book);
92 }
```

Controllers > BooksController.cs > {} EFCorePractice.Controllers > BooksController

```
6 namespace EFCorePractice.Controllers;
10 public class BooksController : ControllerBase

94 [HttpDelete("{id}")]
    0 references
96 public async Task<IActionResult> DeleteBook(
97     int id)
98 {
99     Book? book =
100         await _context.Books.FindAsync(id);
101
102     if (book == null)
103     {
104         return NotFound();
105     }
106
107     _context.Books.Remove(book);
108
109     await _context.SaveChangesAsync();
110
111     return Ok(
112         "Book deleted successfully."
113     );
114 }
```

Output:

localhost:5068/swagger/index.html

PUT /api/Books/{id}

Parameters

Name	Description
id * required	
integer(\$int32)	1
(path)	

Request body

application/json

Edit Value | Schema

```
{
  "bookId": 1,
  "title": "Clean Code",
  "author": "Robert C. Martin"
}
```

Execute Clear

localhost:5068/swagger/index.html

Responses

Curl

```
curl -X 'PUT' \
  'http://localhost:5068/api/Books/1' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "bookId": 1,
    "title": "Clean Code",
    "author": "Robert C. Martin"
  }'
```

Request URL

http://localhost:5068/api/Books/1

Server response

Code	Details
200	Response body <pre>{ "bookId": 1, "title": "Clean Code", "author": "Robert C. Martin" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Thu, 03 Sep 2026 18:04:22 GMT server: Kestrel transfer-encoding: chunked</pre>

Responses

Code	Description	Links
200	OK	No links

Request URL
`http://localhost:5068/api/Books`

Server response

Code	Details
200	Response body <pre>[{ "bookId": 1, "title": "Clean Code", "author": "Robert C. Martin" }, { "bookId": 2, "title": "The Pragmatic Programmer", "author": "Andrew Hunt" }]</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Thu, 03 Sep 2026 18:05:35 GMT server: Kestrel transfer-encoding: chunked</pre>

Responses

Code	Description	Links
200	OK	No links

PUT `/api/Books/{id}`

DELETE `/api/Books/{id}`

GET `/api/Books/author/{authorName}`

DELETE `/api/Books/{id}`

Parameters

Name	Description
<code>id</code> * required <code>integer(\$int32)</code> (path)	2

Execute Clear

Responses

Curl

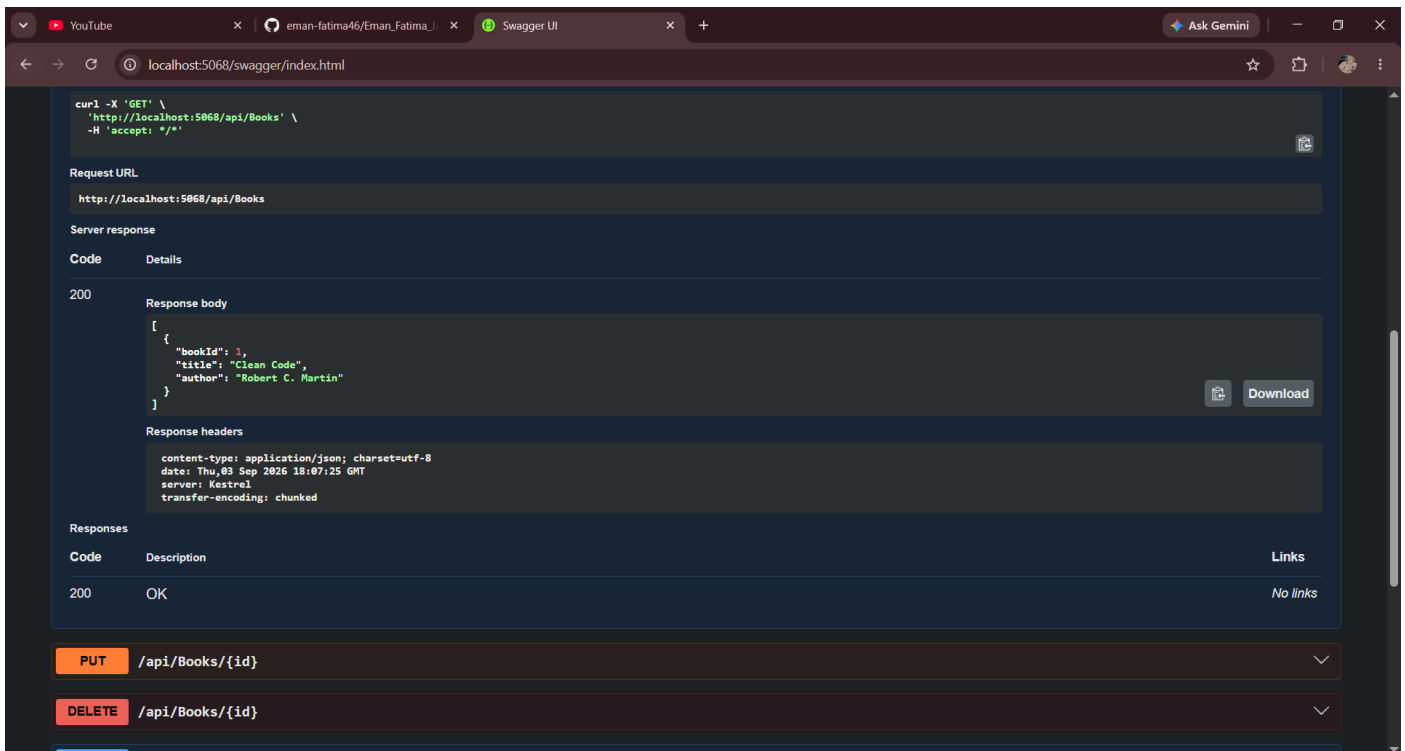
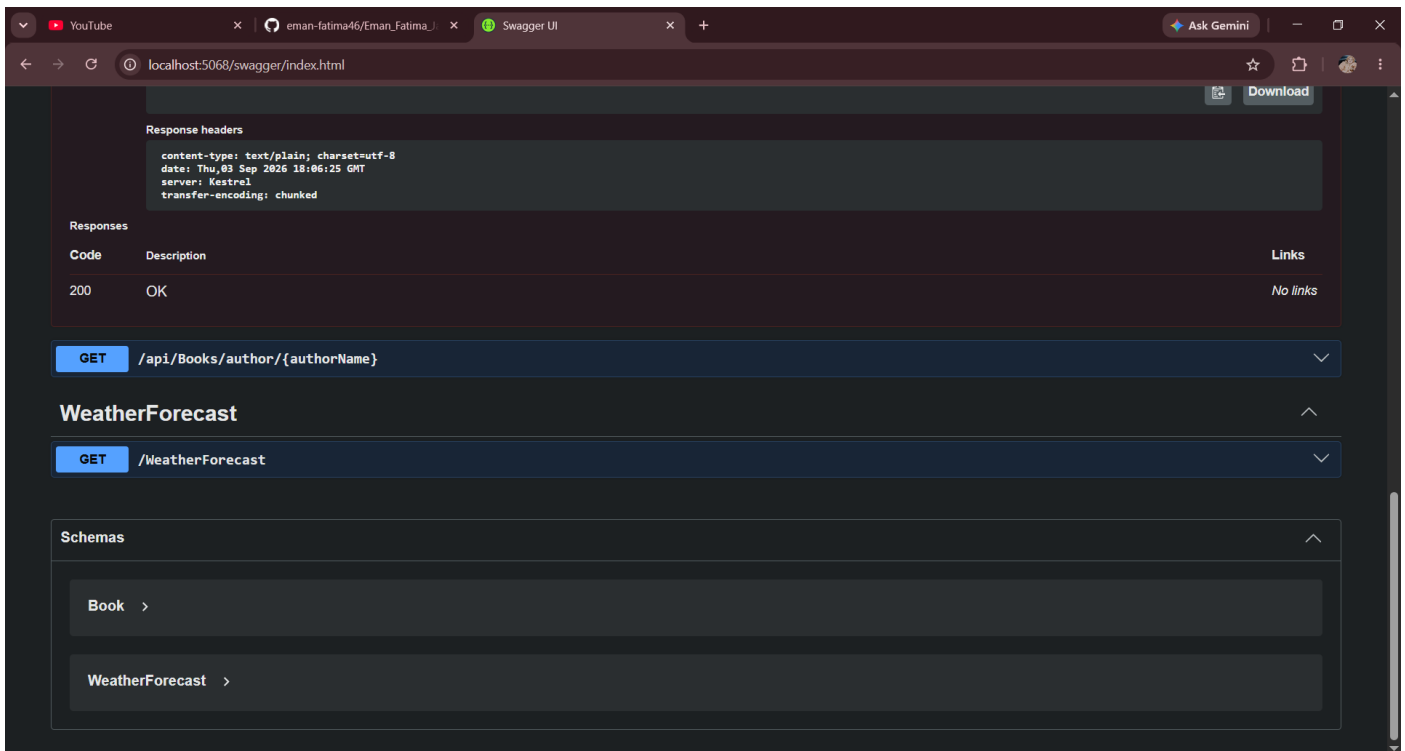
```
curl -X 'DELETE' \
'http://localhost:5068/api/Books/2' \
-H 'accept: */*'
```

Request URL

`http://localhost:5068/api/Books/2`

Server response

Code	Details
200	Response body <pre>Book deleted successfully.</pre>



4. Filter Records Using LINQ

Code:

Controllers > BooksController.cs > {} EFCorePractice.Controllers > BooksController

```
6 namespace EFCorePractice.Controllers;
10 public class BooksController : ControllerBase

117 [HttpGet("author/{authorName}")]
    0 references
118 public async Task<IActionResult> GetBooksByAuthor(
119     string authorName)
120 {
121     List<Book> books =
122         await _context.Books
123             .Where(book =>
124                 book.Author == authorName)
125             .ToListAsync();
126
127     return Ok(books);
128 }
129 }
```

Output:

The screenshot shows the Swagger UI interface for a REST API. The top bar indicates the current endpoint is `GET /api/Books/author/{authorName}`. Below this, the 'Parameters' section shows a required string parameter `authorName` with the value `Robert C. Martin` entered. The 'Execute' button is highlighted in blue. Below the 'Execute' button, the 'Responses' section is expanded, showing the response for status code 200. The response body is a JSON array of books, with the first book having `bookId: 1` and `title: 'Clean Code'`. The response headers are also visible, showing `content-type: application/json; charset=utf-8` and `date: Thu, 03 Sep 2026 18:09:18 GMT`. At the bottom, a table of responses shows a 200 status code with the description 'OK' and no links.

Code	Description	Links
200	OK	No links

PART C: API + DATABASE INTEGRATION

Exercises:-

1. Replace In-Memory Repository With EF Core and Re-Test All API Endpoints

Code:

```
Data > C# LibraryDbContext.cs > {} LibraryApi.Data > LibraryDbContext
1  using LibraryApi.Models;
2  using Microsoft.EntityFrameworkCore;
3
4  namespace LibraryApi.Data;
5
6  7 references
   public class LibraryDbContext : DbContext
7  {
8      0 references
       public LibraryDbContext(
9       DbContextOptions<LibraryDbContext> options)
10      : base(options)
11      {
12      }
13
14      6 references
       public DbSet<Book> Books => Set<Book>();
15  }
```

```
Repositories > C# BookRepository.cs > {} LibraryApi.Repositories > BookRepository > AddAsync(Book) : Task<bool>
1  using LibraryApi.Data;
2  using LibraryApi.Models;
3  using Microsoft.EntityFrameworkCore;
4
5  namespace LibraryApi.Repositories;
6
7  2 references
   public class BookRepository : IBookRepository
8  {
9      10 references
       private readonly LibraryDbContext _context;
10
11      0 references
       public BookRepository(
12       LibraryDbContext context)
13      {
14       _context = context;
15      }
```

```

Repositories > BookRepository.cs > {} LibraryApi.Repositories > BookRepository > AddAsync(Book) : Task<bool>
5  namespace LibraryApi.Repositories;
7  public class BookRepository : IBookRepository
18     public async Task<List<Book>> GetAllAsync()
19     {
20         return await _context.Books
21             .ToListAsync();
22     }
23
24
25     3 references
26     public async Task<Book?> GetByIdAsync(int id)
27     {
28         return await _context.Books
29             .FindAsync(id);
30     }
31
32     2 references
33     public async Task<bool> AddAsync(Book book)
34     {
35         try
36         {
37             await _context.Books.AddAsync(book);
38             await _context.SaveChangesAsync();
39             return true;
40         }
41     }

```

```

Repositories > BookRepository.cs > {} LibraryApi.Repositories > BookRepository > AddAsync(Book) : Task<bool>
5  namespace LibraryApi.Repositories;
7  public class BookRepository : IBookRepository
32     public async Task<bool> AddAsync(Book book)
42     {
43         catch (DbUpdateException ex)
44         {
45             Console.WriteLine("DATABASE ERROR:");
46             Console.WriteLine(ex.Message);
47             if (ex.InnerException != null)
48             {
49                 Console.WriteLine(ex.InnerException.Message);
50             }
51         }
52         return false;
53     }
54
55
56
57     2 references
58     public async Task<bool> UpdateAsync(Book book)
59     {
60         try
61         {
62             Book? existingBook =
63                 await _context.Books.FindAsync(
64                     book.Id
65                 );

```

Repositories > BookRepository.cs > {} LibraryApi.Repositories > BookRepository > AddAsync(Book) : Task<bool>

```
5 namespace LibraryApi.Repositories;
7 public class BookRepository : IBookRepository
57     public async Task<bool> UpdateAsync(Book book)
66     {
67         if (existingBook == null)
68         {
69             return false;
70         }
71         existingBook.Title =
72             book.Title;
73         existingBook.Author =
74             book.Author;
75         existingBook.Category =
76             book.Category;
77         await _context.SaveChangesAsync();
78         return true;
79     }
80     catch (DbUpdateException)
81     {
82         return false;
83     }
84 }
85
86
87
88
89
90
```

Repositories > BookRepository.cs > {} LibraryApi.Repositories > BookRepository > AddAsync(Book) : Task<bool>

```
5 namespace LibraryApi.Repositories;
7 public class BookRepository : IBookRepository
91     public async Task<bool> DeleteAsync(int id)
92     {
93         try
94         {
95             Book? book =
96                 await _context.Books.FindAsync(id);
97             if (book == null)
98             {
99                 return false;
100             }
101             _context.Books.Remove(book);
102             await _context.SaveChangesAsync();
103             return true;
104         }
105         catch (DbUpdateException)
106         {
107             return false;
108         }
109     }
110 }
111
112
113
114
```

Controllers > C# BooksController.cs > {} LibraryApi.Controllers > BooksController

```
1  using LibraryApi.Models;
2  using LibraryApi.Services;
3  using Microsoft.AspNetCore.Mvc;
4
5  namespace LibraryApi.Controllers;
6
7  [ApiController]
8  [Route("api/[controller]")]
9  public class BooksController : ControllerBase
10 {
11     private readonly IBookService _bookService;
12
13     public BooksController(
14         IBookService bookService)
15     {
16         _bookService = bookService;
17     }
18
19     [HttpGet]
20     public async Task<IActionResult> GetAll()
21     {
22         var books =
23             await _bookService
24                 .GetAllBooksAsync();
25         return Ok(books);
26     }
27
28     [HttpGet("{id}")]
29
30
31
```

Controllers > C# BooksController.cs > {} LibraryApi.Controllers > BooksController

```
5 namespace LibraryApi.Controllers;
9 public class BooksController : ControllerBase

32 public async Task<IActionResult>
    1 reference
33     GetById(int id)
34 {
35     Book? book =
36         await _bookService
37             .GetBookByIdAsync(id);
38
39     if (book == null)
40     {
41         return NotFound(
42             $"Book with ID {id} was not found."
43         );
44     }
45
46     return Ok(book);
47 }

48
49
50 [HttpPost]
51 public async Task<IActionResult>
    0 references
52     Create(Book book)
53 {
54     try
55     {
56         bool created =
57             await _bookService
58                 .AddBookAsync(book);
59     }
```

Controllers >  BooksController.cs > { } LibraryApi.Controllers >  BooksController

```
5 namespace LibraryApi.Controllers;
9 public class BooksController : ControllerBase
52     Create(Book book)
60         if (!created)
61         {
62             return BadRequest(
63                 "Book could not be created."
64             );
65         }
66
67         return CreatedAtAction(
68             nameof(GetById),
69             new { id = book.Id },
70             book
71         );
72     }
73     catch (Exception)
74     {
75         return StatusCode(
76             500,
77             "A database error occurred while creating the book."
78         );
79     }
80 }
81
82
83 [HttpPut("{id}")]
84 public async Task<IActionResult>
85     Update(
86         int id,
87         Book book)
```

Controllers > C# BooksController.cs > {} LibraryApi.Controllers > BooksController

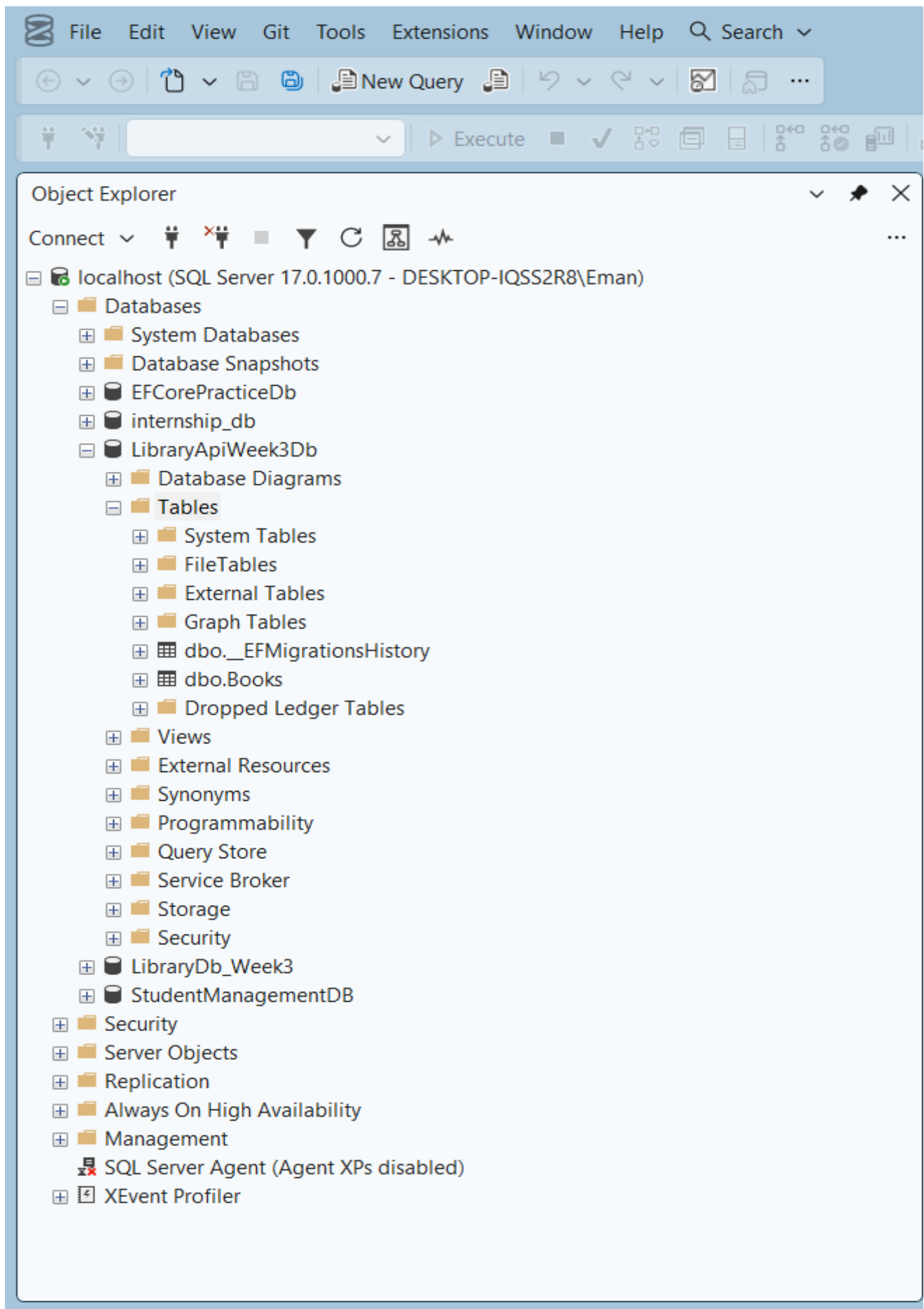
```
5 namespace LibraryApi.Controllers;
9 public class BooksController : ControllerBase
85     Update(
88     {
89         try
90         {
91             bool updated =
92                 await _bookService
93                     .UpdateBookAsync(
94                         id,
95                         book
96                     );
97
98             if (!updated)
99             {
100                 return BadRequest(
101                     "Book could not be updated."
102                 );
103             }
104
105             return Ok(book);
106         }
107         catch (Exception)
108         {
109             return StatusCode(
110                 500,
111                 "A database error occurred while updating the book."
112             );
113         }
114     }
115
116
117     [HttpDelete("{id}")]
118     public async Task<IActionResult>
```


Controllers > C# BooksController.cs > {} LibraryApi.Controllers > BooksController

```
5 namespace LibraryApi.Controllers;
9 public class BooksController : ControllerBase
119     Delete(int id)
120     {
121         Book? existingBook =
122             await _bookService
123                 .GetBookByIdAsync(id);
124
125         if (existingBook == null)
126         {
127             return NotFound(
128                 $"Book with ID {id} was not found."
129             );
130         }
131
132         try
133         {
134             bool deleted =
135                 await _bookService
136                     .DeleteBookAsync(id);
137
138             if (!deleted)
139             {
140                 return StatusCode(
141                     500,
142                     "Book could not be deleted."
143                 );
144             }
145
146             return Ok(
147                 "Book deleted successfully."
148             );
149         }
150         catch (Exception)
```

```
151     {
152         return StatusCode(
153             500,
154             "A database error occurred while deleting the book."
155         );
156     }
157 }
158 }
```

Output:



Swagger UI

localhost:5120/swagger/index.html

Select a definition **LibraryApi v1**

LibraryApi ^{1.0} OAS 3.0

<http://localhost:5120/swagger/v1/swagger.json>

Books

- GET** /api/Books
- POST** /api/Books
- GET** /api/Books/{id}
- PUT** /api/Books/{id}
- DELETE** /api/Books/{id}

WeatherForecast

- GET** /WeatherForecast

Schemas

Swagger UI

localhost:5120/swagger/index.html

GET /api/Books

Parameters

No parameters

Execute **Clear**

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5120/api/Books' \
  -H 'accept: */*'
```

Request URL

<http://localhost:5120/api/Books>

Server response

Code **Details**

200

Response body

```
[
  {
    "id": 4,
    "title": "Clean Code",
    "author": "Robert C. Martin",
    "category": "Programming"
  }
]
```

Download

Response headers

Response headers

```
content-type: application/json; charset=utf-8
date: Sun, 06 Sep 2026 14:01:28 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	OK	No links

Swagger UI

localhost:5120/swagger/index.html

POST /api/Books

Parameters

No parameters

Request body

application/json

Edit Value | Schema

```
{
  "id": 5,
  "title": "Xyz",
  "author": "Robert C. Martin",
  "category": "Programming"
}
```

Execute Clear

Responses

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:5120/api/Books' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 5,
    "title": "Xyz",
    "author": "Robert C. Martin",
    "category": "Programming"
  }'
```

Request URL

http://localhost:5120/api/Books

Server response

Code	Details
201	Response body <pre>{ "id": 5, "title": "Xyz", "author": "Robert C. Martin", "category": "Programming" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Sun, 06 Sep 2026 14:03:10 GMT location: http://localhost:5120/api/Books/5 server: Kestrel transfer-encoding: chunked</pre>

Responses

Code	Description	Links
200	OK	No links

Swagger UI

Ask Gemini

localhost:5120/swagger/index.html

PUT

/api/Books/{id}

Parameters

Cancel

Reset

Name	Description
id * required integer(\$int32) (path)	5

Request body

application/json

Edit Value | Schema

```
{
  "id": 5,
  "title": "Xyz_Updated",
  "author": "Robert C. Martin",
  "category": "Programming"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'PUT' \
  'http://localhost:5120/api/Books/5' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 5,
    "title": "Xyz_Updated",
    "author": "Robert C. Martin",
    "category": "Programming"
  }'
```

Request URL

http://localhost:5120/api/Books/5

Server response

Code	Details
200	Response body <pre>{ "id": 5, "title": "Xyz_Updated", "author": "Robert C. Martin", "category": "Programming" }</pre> Download Response headers <pre>content-type: application/json; charset=utf-8 date: Sun, 06 Sep 2026 14:04:56 GMT server: Kestrel transfer-encoding: chunked</pre>

Responses

Code	Description	Links
200	OK	No links

The screenshot shows the Swagger UI interface for a REST API. The endpoint `DELETE /api/Books/{id}` is selected. The parameter `id` is required and is an integer (path), with the value `5` entered. The `Execute` button is highlighted in blue. Below the parameters, the `Responses` section shows a `200` status code with the response body `Book deleted successfully.`. The `Request URL` is `http://localhost:5120/api/Books/5`. The `Server response` section shows the response headers: `content-type: text/plain; charset=utf-8`, `date: Sun, 06 Sep 2026 14:05:55 GMT`, `server: Kestrel`, and `transfer-encoding: chunked`. The `Responses` table shows a `200` status code with the description `OK` and no links.

2. Handle Missing Book With 404

Code:

```
39         if (book == null)
40         {
41             return NotFound(
42                 $"Book with ID {id} was not found."
43             );
44         }
45
46         return Ok(book);
47     }
```

Output:

Swagger UI interface showing a GET request to `/api/Books/{id}` with the parameter `id` set to `999`. The response is a 404 Not Found error with the message "Book with ID 999 was not found."

Response headers:

```

content-type: text/plain; charset=utf-8
date: Sun, 06 Sep 2026 14:08:52 GMT
server: Kestrel
transfer-encoding: chunked

```

Responses table:

Code	Description	Links
200	OK	No links

3. Handle Database Errors Gracefully

Code:

```

50 [HttpPost]
51 public async Task<IActionResult>
52     Create(Book book)
53     {
54         try
55         {
56             bool created =
57                 await _bookService
58                     .AddBookAsync(book);
59
60             if (!created)
61             {
62                 return BadRequest(
63                     "Book could not be created."
64                 );
65             }
66
67             return CreatedAtAction(
68                 nameof(GetById),
69                 new { id = book.Id },
70                 book
71             );
72         }
73         catch (Exception)
74         {
75             return StatusCode(
76                 500,
77                 "A database error occurred while creating the book."
78             );
79         }
80     }

```

PART D: AUTHENTICATION CONCEPTS

Exercises:-

1. Identify and Decode a JWT

- For this exercise, use this example JWT:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

- Notice the two dots:

Part 1 . Part 2 . Part 3

Part 1: Header

The first part is:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9

When decoded, it contains:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Explanation

- alg tells us the algorithm associated with the token signature.
- typ tells us this token is a JWT.

Part 2: Payload

The second part is:

eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ

When decoded:

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "iat": 1516239022
}
```

Explanation

The payload contains claims.

Here:

sub

means the subject/identifier associated with the token.

name

contains a name.

iat

means when the token was issued.

Part 3: Signature

The final part is:

SflKxwRJSMekKF2QT4fwpMeJf36POk6yJV_adQssw5c

Explanation

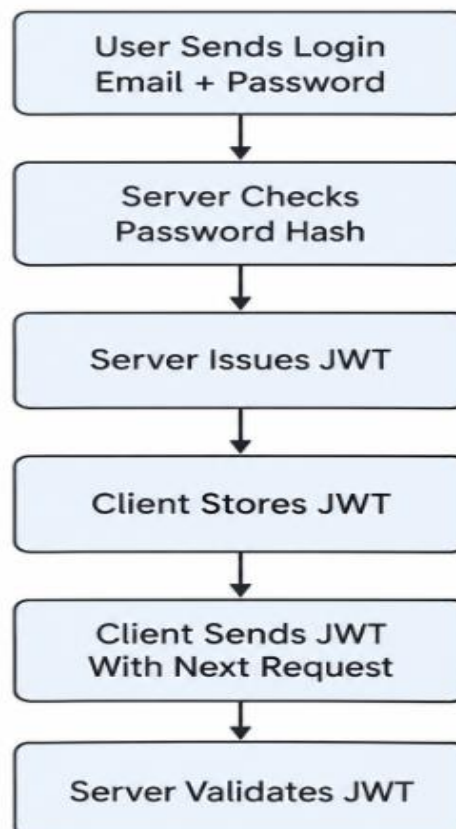
The signature is used by the server to check whether the token has been altered and whether it was signed with the expected secret/key.

A JWT contains three parts separated by dots: Header, Payload, and Signature. I decoded the Header and Payload of an example JWT. The Header showed that the token type was JWT and identified the signing algorithm. The Payload contained claims such as the subject, name, and issue time. The Signature was not decoded because it is used for verifying the integrity of the token.

2. What If Passwords Were Stored as Plain Text?

If passwords were stored as plain text and the database was leaked, anyone who obtained the database could immediately read every user's password. Attackers could use those passwords to access the application and may also try the same passwords on other websites because many people reuse passwords. Passwords should therefore be stored using secure one-way hashing rather than as readable text.

3. Draw the Login and JWT Sequence



After a login request, the server verifies the password against its stored hash. If the credentials are valid, the server issues a JWT. The client stores the token and sends it with later requests. The server validates the token before allowing access.

PART E: ANGULAR API INTEGRATION

Exercises:-

1. Connect BookService to the Real API

Code:

```
LibraryAngular > src > app > services > TS book.service.ts > BookService
1  import { Injectable } from '@angular/core';
2  import { HttpClient } from '@angular/common/http';
3  import { Observable } from 'rxjs';
4
5  import { Book } from '../book';
6  import { environment } from '../../environments/environment';
7
8  @Injectable({
9    providedIn: 'root'
10 })
11 export class BookService {
12
13   private apiUrl =
14     `${environment.apiUrl}/api/Books`;
15
16   constructor(
17     private http: HttpClient
18   ) {
19   }
20
21   getBooks(): Observable<Book[]> {
22     return this.http.get<Book[]>(
23       this.apiUrl
24     );
25   }
}
```

```
LibraryAngular > src > environments > TS environment.ts > environment > apiUrl
1  export const environment = {
2    apiUrl: 'http://localhost:5120'
3  };
```

Output:

Library Management

[Book List](#) [Add Book](#)

Library Books

ID	Title	Author	Category	Actions
4	Clean Code	Robert C. Martin	Programming	Edit Delete

2. Handle API Error When Backend Is Off

Code:

```
LibraryAngular > src > app > book-list > TS book-list.ts > BookList
14  import {
15    |   BookService
16  } from '../services/book.service';
17
18  @Component({
19    selector: 'app-book-list',
20    imports: [RouterLink],
21    templateUrl: './book-list.html',
22    styleUrls: ['./book-list.css']
23  })
24  export class BookList
25    implements OnInit {
26
27    books: Book[] = [];
28
29    isLoading = false;
30
31    errorMessage = '';
```

```

LibraryAngular > src > app > book-list > TS book-list.ts > BookList
24 export class BookList
42   loadBooks(): void {
57     error: (error) => {
58       console.error(error);
59
60       this.errorMessage =
61         'Could not load books. Please make sure the backend API is running.';
62
63       this.isLoading = false;
64     }
65
66   });
67 }

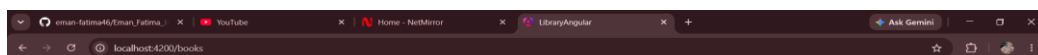
```

```

LibraryAngular > src > app > book-list > book-list.html > ...
1  <h2>Library Books</h2>
2
3  @if (isLoading) {
4
5    <p>
6      Loading...
7    </p>
8
9  }
10
11 @if (errorMessage) {
12
13   <p class="error">
14     {{ errorMessage }}
15   </p>
16
17 }

```

Output:



3. Add Loading State

Code:

LibraryAngular > src > app > book-list > TS book-list.ts > BookList

```
24 export class BookList
29   isLoading = false;
30
31   errorMessage = '';
32
33   constructor(
34     | private bookService: BookService
35   ) {
36   }
37
38   ngOnInit(): void {
39     | this.loadBooks();
40   }
41
42   loadBooks(): void {
43
44     this.isLoading = true;
45
46     this.errorMessage = '';
47
48     this.bookService
49       .getBooks()
50       .subscribe({
51
52         next: (books) => {
53           | this.books = books;
54           | this.isLoading = false;
55         },
56
57         error: (error) => {
58           | console.error(error);
59         }
59       }
59     );
59   }
59 }
```

LibraryAngular > src > app > book-list > <> book-list.html > ...

```
1 <h2>Library Books</h2>
2
3 @if (isLoading) {
4
5   <p>
6     | Loading...
7   </p>
8
9 }
```

Output:

Library Management

[Book List](#) [Add Book](#)

Library Books

Loading...

4. Submit Reactive Form to POST a New Book

Code:

LibraryAngular > src > app > book-form > TS book-form.ts > BookForm

```
32   export class BookForm
128   onSubmit(): void {
176   }
177   else {
178
179     this.bookService
180       .addBook(book)
181       .subscribe({
182
183         next: () => {
184
185           this.isSaving = false;
186
187           this.router.navigate(
188             ['/books']
189           );
190         },
191
192         error: () => {
193
194           this.isSaving = false;
195
196           this.errorMessage =
197             'Could not add the book.';
198         }
199       });
200   }
201 }
202 }
203 }
```

LibraryAngular > src > app > services > TS book.service.ts > BookService

```
11  export class BookService {  
33      addBook(book: Book): Observable<Book> {  
34          return this.http.post<Book>(  
35              this.apiUrl,  
36              book  
37          );  
38      }
```

LibraryAngular > src > app > book-form > book-form.html > h2

```
19  
20  <form  
21      [formGroup]="bookForm"  
22      (ngSubmit)="onSubmit()">>
```

LibraryAngular > src > app > book-form > book-form.html > h2

```
20  <form  
125      <button  
126          type="submit"  
127          [disabled]="isSaving">  
128  
129          @if (isSaving) {  
130              Saving...  
131          }  
132          @else if (isEditMode) {  
133              Update Book  
134          }  
135          @else {  
136              Add Book  
137          }  
138      </button>  
139  
140  
141  </form>
```

Output:

Library Management

[Book List](#) [Add Book](#)

Add Book

Book ID

5

Title

ASP.NET Core Basics

Author

John Smith

Category

Programming

Add Book

Library Management

[Book List](#) [Add Book](#)

Library Books

ID	Title	Author	Category	Actions
4	Clean Code	Robert C. Martin	Programming	Edit Delete
5	ASP.NET Core Basics	John Smith	Programming	Edit Delete

PART F: ENTITY FRAMEWORK CORE

Exercises:-

1. Repository and Branch Workflow

I used Git and GitHub to manage the Week 3 work. Each major part was completed on a separate feature branch and merged into main through a Pull Request. The branches are as:

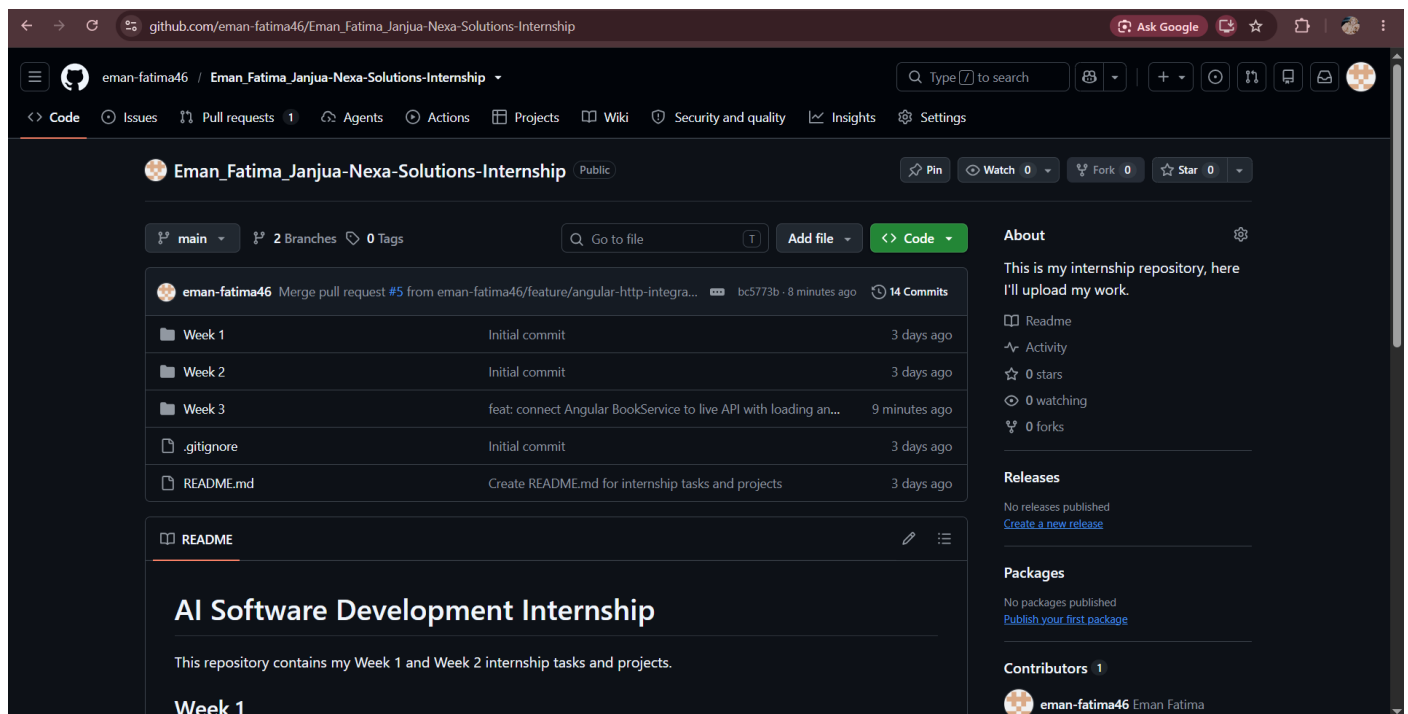
- feature/relational-schema
- feature/ef-core-setup

- feature/api-db-integration
- docs/week3-auth-notes
- feature/angular-http-integration
- feature/ai-summary-script

2. Commit Message Convention

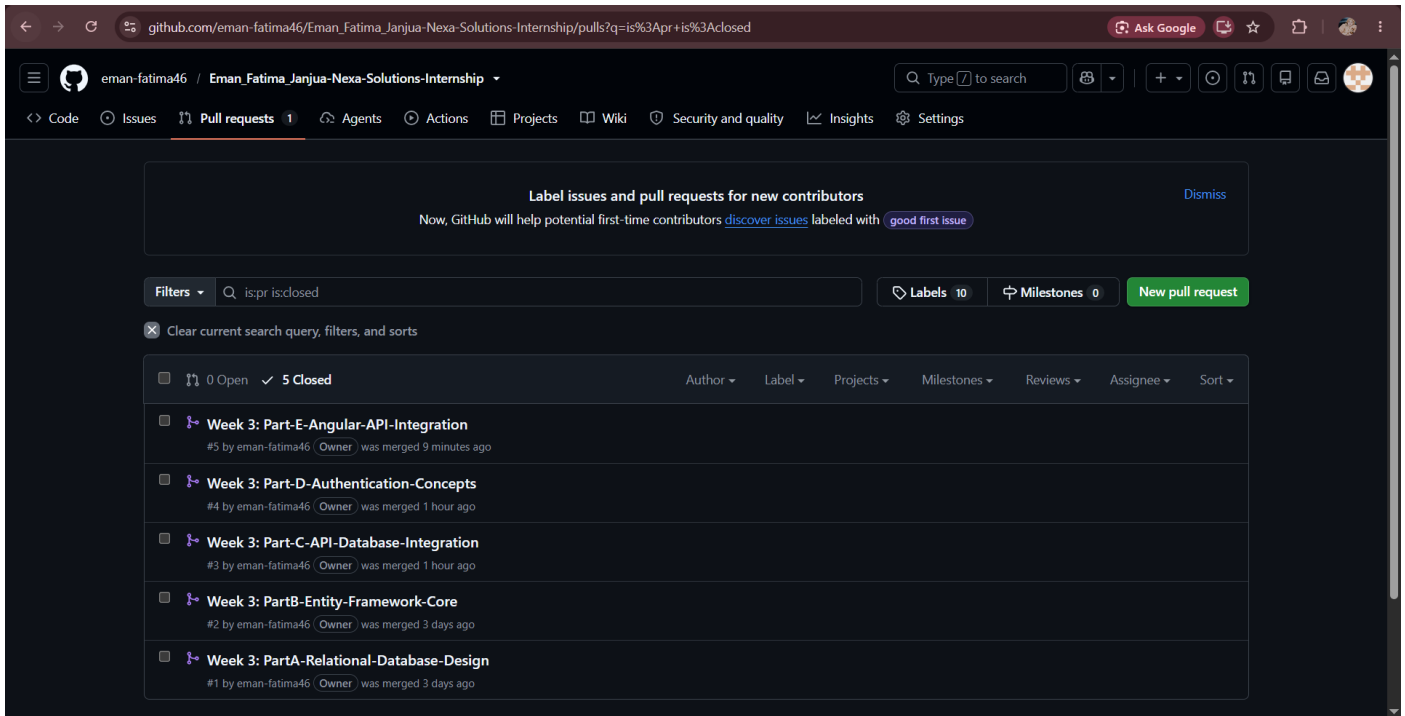
Commit messages were written using prefixes so the repository history is easy to understand. It includes:

```
feat:      new feature
fix:      bug fix
docs:     documentation
chore:    setup/configuration/migrations
refactor: code restructuring
```



3. Pull Request Workflow

After pushing each feature branch, I created a Pull Request into main, reviewed the changed files, and merged the branch after confirming the changes were correct.



4. .gitignore

.gitignore prevents generated files, dependencies, virtual environments, and secret configuration files from being committed to GitHub.

```
bin/  
obj/  
appsettings.Development.json  
node_modules/  
dist/  
.venv/  
__pycache__/  
.env
```

5. README.md

README.md is used as the main documentation page of the repository. It should explain the project, technologies used, and how to run the backend, frontend, and AI script.

6. Secret Handling

Sensitive files such as .env, API keys, passwords, and local development connection settings should never be committed to GitHub. They are excluded using .gitignore.

This is important because the task sheet warns that removing a leaked secret in a later commit is not enough; it remains in Git history.

PART G: AI & PYTHON FOUNDATIONS (KICKOFF)

Exercises:-

1. Run the First AI/Python Script

Code:

```
ai-scripts > .env
1 GROQ_API_KEY=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
```

```
ai-scripts > main.py > ...
1 import os
2
3 from dotenv import load_dotenv
4 from groq import Groq
5
6
7 load_dotenv()
8
9 api_key = os.getenv("GROQ_API_KEY")
10
11 if not api_key:
12     raise ValueError(
13         "GROQ_API_KEY was not found. "
14         "Please add it to the .env file."
15     )
16
17 client = Groq(
18     api_key=api_key
19 )
20
21 response = client.chat.completions.create(
22     model="openai/gpt-oss-20b",
23     messages=[
24         {
25             "role": "user",
26             "content": "Explain what a token is in one paragraph."
27         }
28     ],
29     max_tokens=300
30 )
31
32 print("AI Response:")
33 print(response.choices[0].message.content)
```

Output:

```
(.venv) PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 3\Part6-AI-Python-Foundations\ai-scripts> python main.py
AI Response:
A token is the smallest discrete unit that a language model or a tokenizer processes; it can be a whole word, a part of a word (sub-word or character), or even a punctuation mark, depending on the tokenization scheme. In natural language processing, text is first split into tokens so the model can map each token to a n embedding vector and then learn patterns across sequences. The exact definition of a token varies by context; in programming a token might be a keyword, operator, or identifier; in cryptography a token is a piece of data representing a credential; in conversational AI, tokens count toward the input-output limit. In all cases, a token serves as a standardized, manageable unit that bridges raw data and the algorithm's internal representations.
```

2. Compare Three Prompt Styles

Code:

```
ai-scripts > prompt_experiments.py > ...
1  import os
2
3  from dotenv import load_dotenv
4  from groq import Groq
5
6
7  load_dotenv()
8
9  api_key = os.getenv("GROQ_API_KEY")
10
11 if not api_key:
12     raise ValueError(
13         "GROQ_API_KEY was not found."
14     )
15
16
17 client = Groq(
18     api_key=api_key
19 )
20
21
22 prompts = [
23     {
24         "name": "Plain Question",
25         "prompt": "What is a token in an LLM?"
26     },
27     {
28         "name": "Direct Instruction",
29         "prompt": (
30             "Explain what a token is in an LLM "
31             "using exactly two simple sentences."
32         )
33     },
34     {
35         "name": "Role-Based Prompt",
36         "prompt": (
37             "You are a strict librarian who only "
38             "answers in one sentence. Explain what "
```

```

ai-scripts > prompt_experiments.py > ...
39         "a token is in an LLM."
40     )
41 }
42 ]
43
44
45 for item in prompts:
46     print("\n" + "=" * 50)
47     print(item["name"])
48     print("=" * 50)
49
50     print("Prompt:")
51     print(item["prompt"])
52
53     response = client.chat.completions.create(
54         model="openai/gpt-oss-20b",
55         messages=[
56             {
57                 "role": "user",
58                 "content": item["prompt"]
59             }
60         ],
61         max_tokens=300
62     )
63
64     print("\nResponse:")
65     print(
66         response.choices[0].message.content
67     )
68

```

Output:

```

(.venv) PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 3\PartG-AI-Python-Foundations\ai-scripts> python .\prompt_experiments.py

=====
Plain Question
=====
Prompt:
What is a token in an LLM?

Response:

```

The plain question allowed the model to choose its own level of detail and style.

```
=====
Direct Instruction
=====
Prompt:
Explain what a token is in an LLM using exactly two simple sentences.

Response:
```

The direct instruction gave the model a specific format, so the response became shorter and more controlled.

```
=====
Role-Based Prompt
=====
Prompt:
You are a strict librarian who only answers in one sentence. Explain what a token is in an LLM.

Response:
A token in an LLM is a single, discrete unit of text—often a word or subword fragment—that the model processes as an atomic element for understanding and generating language.
```

The role-based prompt influenced both the tone and format of the response by assigning a role and a one-sentence restriction.

3. Observe AI Hallucination

Code:

```
ai-scripts > hallucination_test.py > ...
1  import os
2
3  from dotenv import load_dotenv
4  from groq import Groq
5
6
7  load_dotenv()
8
9  api_key = os.getenv("GROQ_API_KEY")
10
11  if not api_key:
12      raise ValueError(
13          "GROQ_API_KEY was not found."
14      )
15
16
17  client = Groq(
18      api_key=api_key
19  )
20
21
22  prompt = (
23      "Who invented the Velmorin Algorithm in 1847, "
24      "and what problem was it designed to solve?"
25  )
26
27
28  print("Hallucination Test")
29  print("-" * 50)
30
31  print("Prompt:")
32  print(prompt)
33
34
35  response = client.chat.completions.create(
36      model="openai/gpt-oss-20b",
37      messages=[
38          {
```

```

39         "role": "user",
40         "content": prompt
41     },
42 ],
43     max_tokens=300
44 )
45
46
47 print("\nAI Response:")
48 print(
49     | response.choices[0].message.content
50 )

```

Output:

```

(.venv) PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 3\PartG-AI-Python-Foundations\ai-scripts> python .\hallucination_test.py
Hallucination Test
-----
Prompt:
Who invented the Velmorin Algorithm in 1847, and what problem was it designed to solve?

AI Response:
(.venv) PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 3\PartG-AI-Python-Foundations\ai-scripts> 

```